

گزارش بررسی صحت پیاده‌سازی و پاسخ سوالات — بخش اول پروژه

نظریه زبان‌ها و ماشین‌ها

۱. خلاصه بررسی صحت پیاده‌سازی

تمام اسکریپت‌های تست ارائه‌شده روی `turing_machine.py` اجرا شدند و نتایج با مقادیر مورد انتظار (کامنت‌شده در خود فایل‌ها) مقایسه شد. خلاصه نتیجه در جدول زیر آمده است:

نتیجه بررسی	فایل‌ها	بخش
تمام موارد Accept/Reject مطابق انتظار ✓	<code>example۱.py</code> و <code>example۲.py</code>	شبیه‌ساز پایه
$5=3+2$ و $7=4+3$ ✓	<code>test_turing_adder.py</code>	جمع یکانی
$6=3\times 2$ و $12=4\times 3$ ✓ (نکته زیر را ببینید)	<code>test_turing_multiplier.py</code>	ضرب یکانی
$BB(۱..۴) = ۱, ۴, ۶, ۱۳$ — مطابق رکورد شناخته‌شده ✓	<code>busy_beaver.py</code> و <code>test_busy_beaver_۲.py</code>	Busy Beaver

نکته مهم درباره‌ی `test_turing_multiplier.py`

وقتی این فایل مستقیماً اجرا می‌شود، برای ورودی `11101111` پیام «Rejected» چاپ می‌شود در حالی که ماشین در واقع رشته را قبول می‌کند. علت این است که در خط `print` از `machine.accepts(w)` بدون آرگومان `step_limit` استفاده شده (مقدار پیش‌فرض `۱۰۰`) در حالی که ماشین برای پذیرش این ورودی به بیش از `۱۰۰` گام نیاز دارد؛ در نتیجه `accepts` مقدار `None` برمی‌گرداند که در شرط `if` مانند `False` رفتار می‌کند. خط بعدی که از `machine.debug(w, step_limit=۱۰۰۰)` استفاده می‌کند، درست نتیجه‌ی `Accept` و نوار نهایی `111111111111` (۱۲ عدد ۱، برابر ۴×۳) را نشان می‌دهد. این باگ در خود اسکریپت تست است، نه در `turing_machine.py`. برای رفع آن کافی است در خط چاپ نیز `step_limit` را افزایش داده شود: `machine.accepts(w, step_limit=۱۰۰۰)`.

۲. پاسخ سوالات بخش اول — شبیه‌ساز ماشین تورینگ

الف) لیست حالت‌های هر ماشین

تابع `print_states(transitions)` که در ابتدای هر اسکریپت تست `example۱.py` و `example۲.py` فراخوانی می‌شود، دقیقاً همین کار را انجام می‌دهد و تعداد و نام تمام حالت‌های موجود در دیکشنری `transitions` را استخراج و چاپ می‌کند. برای `example۱`، `q۰`، `#`، `saw` حالت `q۰`، `##`، `qa` و برای `example۲`، `۹` حالت گزارش می‌شود.

ب) نمایش پیکربندی (configuration) به ازای هر انتقال

متد `debug()` که در انتهای هر اسکریپت با `machine.debug(w)` فراخوانی می‌شود، در هر گام محاسبه یک خط با فرمت `state left[symbol]right` چاپ می‌کند که دقیقاً پیکربندی کامل ماشین (حالت فعلی، محتوای سمت چپ و راست نوار و نماد زیر هد) را در همان انتقال نشان می‌دهد. بنابراین این خواسته از قبل در خروجی موجود است و نیازی به تغییر اضافه نیست.

ج) توضیح محاسبات هر ماشین نمونه

- `example۱.py`: این ماشین فقط رشته `##` را قبول می‌کند؛ هر ورودی دیگری (کوتاه‌تر، بلندتر یا با کاراکتر غیر از `#`) را رد می‌کند.
- `example۲.py`: بررسی می‌کند که دو طرف نوار (قبل و بعد از `#`) دقیقاً یک رشته یکسان از `0` و `1` باشند؛ یعنی رشته‌ای به شکل `w#w` را قبول می‌کند (با علامت‌گذاری کاراکترهای بررسی‌شده به صورت `X`).

۳. پاسخ سوالات بخش دوم — محاسبات ریاضی

الف) جمع یکانی (`test_turing_adder.py`)

دو عدد به صورت یکانی (رشته‌ای از `1`) که با یک `0` جدا شده‌اند را دریافت می‌کند. ماشین از روی `1`های عدد اول عبور می‌کند، `0` جداکننده را به `1` تبدیل می‌کند، تا انتهای عدد دوم می‌رود و آخرین `1` اضافه را پاک می‌کند. نتیجه، رشته‌ای واحد از `1` به طول مجموع دو عدد است.

ب) ضرب یکانی (`test_turing_multiplier.py`)

با علامت‌گذاری هر `1` از عدد اول (به `X`)، به ازای هر یک از آن‌ها یک بار کامل از تمام `1`های عدد دوم عبور می‌کند (که موقتاً به `Y` علامت‌گذاری می‌شوند) و برای هر جفت، یک نماد `A` در انتهای نوار اضافه می‌کند. پس از پردازش کامل عدد اول، نوار پاک‌سازی شده و `A`ها به `1` نهایی تبدیل می‌شوند؛ نتیجه رشته‌ای از `1` به طول حاصل ضرب دو عدد است.

۴. پاسخ سوالات بخش سوم — Busy Beaver

الف) چرا یافتن برنامه‌های Busy Beaver ساده نیست

پیدا کردن برنامه‌ی بهینه‌ی Busy Beaver برای n حالت مستلزم بررسی رفتار تمام ماشین‌های تورینگ ممکن با n حالت است. اما برای دانستن این که یک ماشین مفروض روزی متوقف می‌شود یا برای همیشه اجرا می‌ماند (تا بشود مطمئن شد آیا کاندیدای معتبری برای بیشینه است)، باید مسئله‌ی توقف (Halting Problem) را حل کرد که طبق قضیه‌ی تورینگ، به صورت الگوریتمی و برای حالت کلی غیرقابل حل (undecidable) است. به همین دلیل تابع Busy Beaver خودش نیز محاسبه‌ناپذیر (non-computable) است: با افزایش n ، مقدار $\Sigma(n)$ آن قدر سریع رشد می‌کند که هیچ الگوریتم قابل محاسبه‌ای نمی‌تواند آن را برای همه‌ی n پیش‌بینی کند، چون چنین الگوریتمی می‌توانست مسئله‌ی توقف را نیز حل کند.

ب) پشتیبانی از نوار نامتناهی دوطرفه

در `turing_machine.py`، نوار با دو لیست جدا (`tape_left` و `tape_right`) پیاده‌سازی شده است. هنگام حرکت به راست (R)، نماد فعلی به انتهای `tape_left` اضافه و نماد بعدی از ابتدای `tape_right` خوانده می‌شود؛ هنگام حرکت به چپ (L) برعکس این کار انجام می‌شود. اگر هر کدام از این لیست‌ها خالی باشند، به جای خطا، نماد `blank_symbol` تولید می‌شود. به این ترتیب نوار در هر دو جهت به صورت داینامیک و بدون محدودیت گسترش می‌یابد، که پیش‌نیاز اجرای درست برنامه‌های Busy Beaver (که معمولاً به هر دو سمت حرکت می‌کنند) است.

ج) چرا استفاده از generator سودمند است

متد `run()` به عنوان generator نوشته شده تا کنترل اجرا به فراخواننده منتقل شود: هر بار که `next()` صدا زده می‌شود، ماشین دقیقاً یک گام محاسبه را انجام داده و متوقف (`suspend`) می‌شود. این ویژگی چند مزیت دارد: (۱) امکان اجرای گام به گام و دیباگ کردن محاسبات پیچیده مانند Busy Beaver را فراهم می‌کند، (۲) چون بسیاری از ماشین‌های تورینگ (به خصوص در مسئله‌ی Busy Beaver) ممکن است هرگز متوقف نشوند، generator اجازه می‌دهد بدون گیر افتادن در یک حلقه‌ی بی‌نهایت، تعداد گام مشخصی (`step_limit`) بررسی و سپس تصمیم‌گیری شود، و (۳) از نظر حافظه کارآمدتر است چون تاریخچه‌ی کامل اجرا از قبل تولید و ذخیره نمی‌شود، بلکه هر پیکربندی فقط در لحظه‌ی نیاز محاسبه می‌شود (`lazy evaluation`).

د) نتیجه‌ی اجرای برنامه‌ی ۲ کارتی

با اجرای برنامه‌ی ۲ کارتی نمونه (`test_busy_beaver_۲.py` / `busy_beaver.py`) روی نوار خالی، ماشین پس از ۶ گام متوقف می‌شود و ۴ عدد نماد 1 روی نوار باقی می‌ماند.

ه) مقایسه با برنامه‌های ۲ کارتی شناخته شده

طبق مراجع شناخته‌شده‌ی Busy Beaver، مقدار بهینه برای ۲ حالت $\Sigma(2) = 4$ است. نتیجه‌ی به‌دست‌آمده (۴ عدد 1) دقیقاً با این مقدار بهینه برابر است؛ یعنی این برنامه‌ی ۲ کارتی از قبل بهینه است و نمی‌توان با ۲ حالت به نتیجه‌ی بهتری رسید.

و) برنامه‌های ۳ و ۴ کارتی

برنامه‌ی ۳ کارتی ارائه‌شده در فایل‌ها پس از ۱۴ گام متوقف می‌شود و ۶ عدد 1 تولید می‌کند؛ برنامه‌ی ۴ کارتی پس از ۱۰۷ گام متوقف می‌شود و ۱۳ عدد 1 تولید می‌کند. هر دو مقدار دقیقاً برابر با بیشینه‌ی شناخته‌شده ($\Sigma(3)=6$ و $\Sigma(4)=13$) هستند و در نتیجه هر دو بیشتر از نتیجه‌ی برنامه‌ی ۲ کارتی (۴ عدد 1) هستند — همان‌طور که سوال خواسته بود.

م) برنامه‌ی ۵ کارتی جدید

نکته: برنامه‌ی bbeaver5 که در فایل test_busy_beaver_۲.py آمده، در واقع همان برنامه‌ی چمپیون شناخته‌شده‌ی ۵ حالت (کشف‌شده توسط Marxen و Buntrock) است که ۴۰۹۸ عدد 1 در 47,176,870 گام تولید می‌کند؛ این یک برنامه‌ی «جدید» نیست بلکه بهترین برنامه‌ی شناخته‌شده است.

ک) مقایسه با Busy Beaver Wiki

بر اساس مراجع شناخته‌شده (از جمله bbchallenge.org و Wikipedia صفحه‌ی Busy Beaver)، بیشینه‌های اثبات‌شده یا شناخته‌شده به این صورت‌اند: $\Sigma(1)=1$ ، $\Sigma(2)=4$ ، $\Sigma(3)=6$ ، $\Sigma(4)=13$ ، $\Sigma(5)=4098$ (اثبات‌شده در سال ۲۰۲۱ توسط پروژه‌ی bbchallenge). برای $\Sigma(6)$ فقط کران پایین شناخته شده و عددی نجومی (بیش از 36534^{10}) است که در کامنت فایل busy_beaver.py هم به آن اشاره شده. برنامه‌های ۲، ۳ و ۴ کارتی پیاده‌سازی‌شده در این پروژه دقیقاً با بهترین مقادیر شناخته‌شده مطابقت دارند و بهینه هستند.